



INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA EN TECNOLOGÍAS AVANZADAS

Ingeniería Telemática Bases de Datos Distribuidas

Tarea 4:

- **Docente:** De la Cruz Sosa Carlos
- **Alumnos:** De Jesús Lucio Oscar Daniel
Pérez Iturbe Carolina
- **Grupo:** 3TM3
- **Fecha de entrega:** 09 de junio 2025
- **Ciclo escolar:** 2025-2



Tarea 4.

Consulta: *Listar el producto más vendido de cada una de las categorías registradas en la base de datos*

Consulta 1:.

```
WITH VentasProducto AS (  
    SELECT  
        pc.Name AS Categoria,  
        p.ProductID,  
        p.Name AS Producto,  
        SUM(od.OrderQty) AS CantidadVendida  
    FROM SalesorderDetail od  
    JOIN Product p ON od.ProductID = p.ProductID  
    JOIN ProductSubcategory ps ON p.ProductSubcategoryID =  
ps.ProductSubcategoryID  
    JOIN ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID  
    GROUP BY pc.Name, p.ProductID, p.Name),  
ProductoMaximo AS (  
    SELECT  
        Categoria,  
        MAX(CantidadVendida) AS MaxCantidad  
    FROM VentasProducto  
    GROUP BY Categoria)  
SELECT vp.Categoria, vp.Producto, vp.CantidadVendida  
FROM VentasProducto vp  
JOIN ProductoMaximo pm  
    ON vp.Categoria = pm.Categoria AND vp.CantidadVendida = pm.MaxCantidad  
ORDER BY vp.Categoria
```

Se le paso a la IA el siguiente Prom para que nos ayudara a optimizarla:

Tengo la siguiente consulta:

Consulta: Listar el producto más vendido de cada una de las categorías registradas en la base de datos.

```
WITH VentasProducto AS (  
    SELECT  
        pc.Name AS Categoria,  
        p.ProductID,  
        p.Name AS Producto,  
        SUM(od.OrderQty) AS CantidadVendida  
    FROM SalesorderDetail od  
    JOIN Product p ON od.ProductID = p.ProductID  
    JOIN ProductSubcategory ps ON p.ProductSubcategoryID =  
ps.ProductSubcategoryID  
    JOIN ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID  
    GROUP BY pc.Name, p.ProductID, p.Name),
```

```
ProductoMaximo AS (  
    SELECT  
        Categoria,  
        MAX(CantidadVendida) AS MaxCantidad  
    FROM VentasProducto  
    GROUP BY Categoria)  
SELECT vp.Categoria, vp.Producto, vp.CantidadVendida  
FROM VentasProducto vp  
JOIN ProductoMaximo pm  
    ON vp.Categoria = pm.Categoria AND vp.CantidadVendida = pm.MaxCantidad  
ORDER BY vp.Categoria
```

Esa fue mi manera de resolverla, pero existe alguna manera que sea más sencilla corta, y que en cuanto a planes de ejecución su ejecución sea mejor.

La respuesta que nos dio es la siguiente:

Tu consulta actual es **correcta** y estructurada, pero efectivamente, puede **optimizarse y simplificarse**. El enfoque que estás usando —una CTE con agregación seguida de otra

CTE para obtener el máximo por categoría— es claro y mantenible, pero podría evaluarse más eficientemente si se usa una técnica que evite la doble agregación.

✓ Versión optimizada usando `WITH TIES` (más sencilla y eficiente en muchos motores SQL como SQL Server y PostgreSQL):

```
sql
SELECT TOP 1 WITH TIES
    pc.Name AS Categoria,
    p.Name AS Producto,
    SUM(od.OrderQty) AS CantidadVendida
FROM SalesOrderDetail od
JOIN Product p ON od.ProductID = p.ProductID
JOIN ProductSubcategory ps ON p.ProductSubcategoryID =
ps.ProductSubcategoryID
JOIN ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID
GROUP BY pc.Name, p.Name
ORDER BY pc.Name, SUM(od.OrderQty) DESC;
```

🔍 ¿Por qué es mejor?

- **Más corta y legible:** Elimina la necesidad de dos CTEs (`VentasProducto` y `ProductoMaximo`).
 - **Más eficiente** en muchos motores SQL:
 - Evita hacer un `JOIN` entre una tabla de máximos y la tabla completa de ventas por producto.
 - `TOP 1 WITH TIES` con `ORDER BY` por categoría te da directamente **los productos con más ventas por categoría**, incluso si hay empates.
-

✓ Resultado

Esta consulta te devolverá:

- Una fila por categoría, con el producto más vendido.
 - En caso de empate en la cantidad vendida en una misma categoría, devolverá **todos** los productos empatados (gracias a `WITH TIES`).
-

📌 Nota importante

- `TOP 1 WITH TIES` es **específico de SQL Server y PostgreSQL** (como `FETCH FIRST 1 ROWS WITH TIES`).
- Si usas **MySQL**, no funciona igual. Para MySQL habría que usar subconsultas o ventanas.

¿Y si hay empates no deseados?

Consulta 2:

Si quieres solo un producto (sin empates) por categoría:

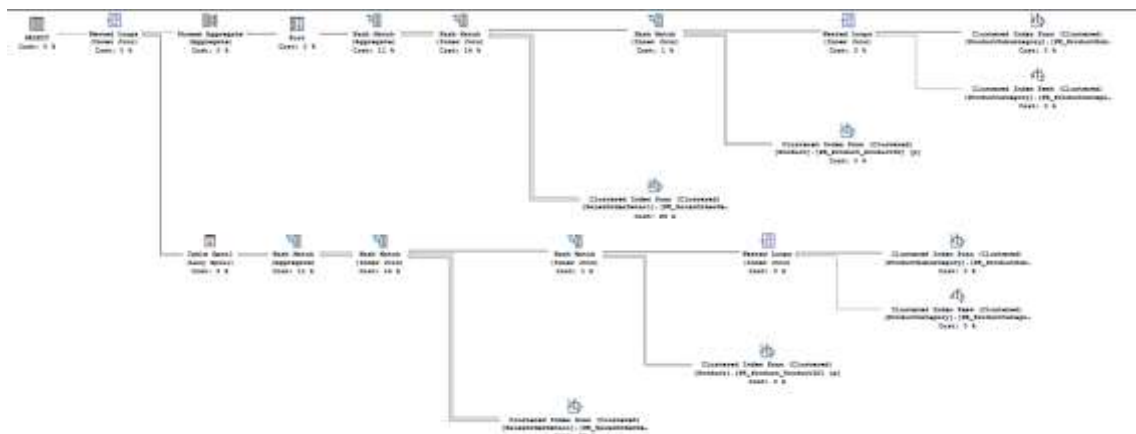
```
sql
SELECT Categoria, Producto, CantidadVendida
FROM (
    SELECT
        pc.Name AS Categoria,
        p.Name AS Producto,
        SUM(od.OrderQty) AS CantidadVendida,
        ROW_NUMBER() OVER (PARTITION BY pc.Name ORDER BY
SUM(od.OrderQty) DESC) AS rn
    FROM SalesOrderDetail od
    JOIN Product p ON od.ProductID = p.ProductID
    JOIN ProductSubcategory ps ON p.ProductSubcategoryID =
ps.ProductSubcategoryID
    JOIN ProductCategory pc ON ps.ProductCategoryID =
pc.ProductCategoryID
    GROUP BY pc.Name, p.Name
) t
WHERE rn = 1;
```

Esta versión usa ROW_NUMBER y da solo un producto (el más vendido) por categoría sin empates.

Conclusiones:

Al llevarlos a la práctica, es decir ejecutarlos en SQL Server nos dimos cuenta que la forma en que funciona de la misma manera que nuestra consulta es la segunda ya que con la primera que nos genera solo nos da una categoría, como se puede ver en el código si es mas compacto y sencillo que la consulta que teníamos y al revisar los planes de ejecución nos dimos cuenta que también cambian:

PLAN DE EJECUCION CONSULTA 1:



PLAN DE EJECUCION CONSULTA 2:

